AI Alignment : Reinforcement Learning from Human Feedback

From RL Introduction to RLHF and DPO

Satya Krishna Pothapragada

March 18, 2026

Contents

1 Language Models **4**

2 AI Alignment: Core Motivation **4**
2.1 Reinforcement Learning as the Core Mechanism 4

3 Reinforcement Learning Basics **4**
3.1 Markov Property 4
3.2 Mathematical Formulation: Markov Decision Process (MDP) 5
3.3 Cumulative Discounted Return 5
3.4 Value Functions 5
3.4.1 State-Value Function 6
3.4.2 Action-Value Function (Q-Function) 6
3.4.3 Relationship Between V and Q 6
3.5 Two Fundamental Philosophies in RL 6
3.6 Q-Learning Update Rule 7
3.7 Trajectory Probability 7

4 Bellman's Value Function Derivation **7**
4.1 Step 1: Start from the Definition 7
4.2 Step 2: Split the Expectation 7
4.3 Step 3: Condition on the Action Taken 8
4.4 Step 4: Expand the Inner Expectation 8
4.5 Step 5: Assemble the Full Bellman Equation 8
4.6 Step 6: Bellman Optimality Equation 8
4.7 Step 7: Policy-Based Methods — Objective Function 9

5 Big Picture: Alignment Pipeline **9**

6 Reward Model **9**
6.1 Dataset Construction 9
6.2 Reward Objective 10

7 RLHF: Policy Gradient Optimisation — REINFORCE **10**
7.1 Objective . . . 10
7.2 Policy Gradient Theorem . . . 10

8 RLHF: Variance Reduction Techniques **11**
8.1 First Problem with Gradient Policy Optimisation: High Variance . . . 11
8.2 Technique 1: Rewards-to-Go . . . 11
8.3 Technique 2: Baseline Subtraction . . . 12
8.4 Advantage Function . . . 12
8.5 Deriving the Reduced Form of $A(s,t)$: TD Residual . . . 12
8.6 k-Step Advantage Generalisation . . . 13
8.7 Generalised Advantage Estimation (GAE) . . . 13

9 Off-Policy Learning **14**
9.1 Second Problem with Gradient Policy Optimisation: On-Policy Learning 14
9.2 Motivation for Off-Policy Learning . . . 14
9.3 Off-Policy Learning Workflow in the RLHF Setting . . . 15

10 Trust Region Policy Optimisation (TRPO) **15**
10.1 Problems with Off-Policy Learning . . . 15
10.2 Trust Region Intuition . . . 15
10.3 TRPO Objective . . . 15
10.4 Full Derivation of the TRPO Update Step . . . 16
10.4.1 Step 1: First-Order Expansion of L . . . 16
10.4.2 Step 2: Second-Order Approximation of the KL Constraint . . . 16
10.4.3 Step 3: Lagrangian Formulation . . . 16
10.4.4 Step 4: Taking the Gradient and Setting to Zero . . . 16
10.4.5 Step 5: Solving for λ via the Equality Constraint . . . 16
10.4.6 Step 6: Final TRPO Update Step . . . 17

11 Proximal Policy Optimisation (PPO) **17**
11.1 Motivation . . . 17
11.2 PPO Clipped Objective . . . 17
11.3 Intuition and Full Case Analysis . . . 17

12 Reward Hacking in RLHF **18**
12.1 The Problem . . . 18
12.2 Reward Model Exploitation . . . 19
12.3 Refined RLHF Objective to Tackle Reward Hacking . . . 19
12.4 Plugging into PPO . . . 19

13 Direct Preference Optimisation (DPO) **20**
13.1 Motivation . . . 20
13.2 Bradley–Terry Model . . . 20
13.3 Deriving the Analytical Solution to the RLHF Objective . . . 20
13.3.1 Step 1: Expand the Objective . . . 20
13.3.2 Step 2: Define an Unnormalised Density . . . 21
13.3.3 Step 3: Apply Gibbs' Inequality . . . 21
13.4 Recovering $r(x,y)$ and Deriving the DPO Loss Function . . . 22

14 Summary: RLHF vs. DPO **22**

1 Language Models

A **language model** is a probabilistic model that assigns probabilities to sequences of words. In practice, a language model allows us to compute the following:

$$P(\text{Next token} \mid \text{input prompt})$$

Example:

$$P(\cdots \mid \texttt{``Where is Shanghai?''})$$

Language models are fundamentally *next-token predictors*; through autoregression they generate words that form sentences, paragraphs, etc.

$$\text{prompt} \longrightarrow \text{tokens} \longrightarrow \text{output}$$

2 AI Alignment: Core Motivation

An LLM pre-trained on a huge amount of data only learns to predict the next token; it does *not* understand the user's intent. We therefore need to **align** the LLM to:

- Behave as a good assistant.
- Comply with human values and ethics (e.g. no profanity, no racist expressions).

Definition 2.1 (Goal of AI Alignment)**.** Align a model's behaviour with a *desired* behaviour.

2.1 Reinforcement Learning as the Core Mechanism

The alignment problem is framed as a Reinforcement Learning (RL) problem:

RL Concept	**LLM Equivalent**
State $s \in \mathcal{S}$	Prompt
Action $a \in \mathcal{A}$	Next-token prediction
Environment	The LLM itself
Reward r	Human preference score

The agent's goal is to learn how to take decisions in a given environment so as to **maximise cumulative reward**.

3 Reinforcement Learning Basics

3.1 Markov Property

Before defining the MDP, we first state the fundamental property that underlies all of RL:

Definition 3.1 (Markov Property)**.**

$$P(s_{t+1} \mid s_t,\ \text{history}) = P(s_{t+1} \mid s_t, a_t)$$

The **current state is sufficient** to predict the next state — the full history is not needed.

Intuition: No matter what happened before state s_t, the distribution over the next state s_{t+1} depends *only* on where we are right now and what action we take. The past is irrelevant once the present is known.

3.2 Mathematical Formulation: Markov Decision Process (MDP)

Definition 3.2 (MDP)**.** An MDP is a tuple $\mathcal{M} = (\mathcal{S},\ \mathcal{A},\ P,\ R,\ \gamma)$ where each element is defined as:

- $\mathcal{S}$: **State space** — the set of all possible states the agent can be in. In LLMs, this is the prompt/context.
- $\mathcal{A}$: **Action space** — the set of all possible actions. In LLMs, this is the next token prediction.
- $P(s' \mid s, a)$: **Transition dynamics** — the probability of moving to state s' given current state s and action a.
- $R(s, a)$: **Reward function** — the immediate scalar reward received after taking action a in state s.
- $\gamma \in [0, 1]$: **Discount factor** — controls how much we care about future rewards vs. immediate rewards.

The core **loop** of the MDP is:

$$\text{State } s \longrightarrow \text{Action } a \longrightarrow \text{Reward } r \longrightarrow \text{Next State } s' \longrightarrow \cdots$$

The agent's goal is to learn how to take decisions intelligently in a given environment so as to **maximise the cumulative reward**.

3.3 Cumulative Discounted Return

At each timestep t, the agent receives a reward r_t. We do not just care about the immediate reward — we care about the **sum of all future rewards**. However, rewards far in the future are worth less than immediate rewards, so we discount them by γ^k:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

We can also write this recursively, which will be important later:

$$G_t = r_t + \gamma G_{t+1}$$

3.4 Value Functions

We want to quantify how good it is to be in a given state, or to take a given action in a given state. This is done via **value functions**.

3.4.1 State-Value Function

The **state-value function** $V^\pi(s)$ is the expected cumulative return starting from state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

Intuition: $V^\pi(s)$ answers the question — "on average, how much reward can I expect to collect from state s, if I follow policy π?"

3.4.2 Action-Value Function (Q-Function)

The **action-value function** $Q^\pi(s, a)$ is the expected cumulative return starting from state s, taking action a, and then following policy π thereafter:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s,\ a_t = a]$$

Intuition: $Q^\pi(s, a)$ answers — "how good is it to take action a in state s?"

3.4.3 Relationship Between V and Q

The value function $V^\pi(s)$ can be written in terms of the Q-function by *averaging over all actions weighted by the policy*:

$$V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s, a)$$

The **goal** of the agent is to maximise $\mathbb{E}[G_t]$, i.e. find the optimal V^* and Q^*.

3.5 Two Fundamental Philosophies in RL

There are two core approaches to solving the RL problem:

1. **Value-Based Methods** — Derive a policy *from* the value function. The policy is **deterministic**:

   $$\pi(s) = \arg\max_a Q(s, a)$$

   We learn what actions are good, then derive the policy directly from that. The weights of a neural network learn to produce good action values, and the policy is extracted as:

   $$a = \pi(s) \to \arg\max_a Q(s, a)$$

2. **Policy-Based Methods** — Directly learn a **stochastic** policy $\pi_\theta(a \mid s)$. Instead of learning values first and deriving a policy, we parameterise the policy directly and optimise it. The objective is:

   $$\max_\theta\ J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

3.6 Q-Learning Update Rule

For value-based methods, the Q-function is updated iteratively using the **Bellman backup**:

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{TD target}} - \underbrace{Q(s,a)}_{\text{current estimate}}\Big]$$

Intuition: We move the current estimate $Q(s,a)$ a small step α towards the bootstrapped target $r + \gamma \max_{a'} Q(s',a')$. Over many iterations, Q converges to Q^*.

3.7 Trajectory Probability

A **trajectory** is a full sequence of states and actions:

$$\tau = (s_0, a_0, s_1, a_1, s_2, a_2, \ldots)$$

The probability of a trajectory τ under policy π is the product of the initial state probability, all transition probabilities, and all action probabilities:

$$p(\tau \mid \pi) = p_0(s_0) \prod_{t=0}^{T} P(s_{t+1} \mid s_t, a_t)\, \pi(a_t \mid s_t)$$

The total return along a trajectory is:

$$R(\tau) = \sum_{t=0}^{T} \gamma^t r_t$$

4 Bellman's Value Function Derivation

We now derive the **Bellman Expectation Equation** step by step, starting purely from the definition of $V^\pi(s)$.

4.1 Step 1: Start from the Definition

Recall:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

and the recursive definition of the return:

$$G_t = r_t + \gamma G_{t+1}$$

Substituting this into the value function:

$$V^\pi(s) = \mathbb{E}_\pi[r_t + \gamma G_{t+1} \mid s_t = s]$$

4.2 Step 2: Split the Expectation

Using linearity of expectation, we split the two terms:

$$V^\pi(s) = \mathbb{E}_\pi[r_t \mid s_t = s] + \gamma\, \mathbb{E}_\pi[G_{t+1} \mid s_t = s]$$

4.3 Step 3: Condition on the Action Taken

The action a_t is drawn from the policy $\pi(a \mid s)$. We introduce the action by summing over all possible actions:

$$V^\pi(s) = \sum_a \pi(a \mid s)\, \mathbb{E}_\pi[r_t + \gamma G_{t+1} \mid s_t = s,\ a_t = a]$$

4.4 Step 4: Expand the Inner Expectation

For a fixed $(s_t = s,\ a_t = a)$, we can expand the inner expectation. The immediate reward $R(s, a)$ is a deterministic function of (s, a), so:

$$\mathbb{E}_\pi[r_t \mid s_t = s,\ a_t = a] = R(s, a)$$

For the future return term, we use the **law of iterated expectations** — condition on the next state s':

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s,\ a_t = a] = \sum_{s'} P(s' \mid s, a)\, \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s']$$

But by definition:

$$\mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] = V^\pi(s')$$

So:

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s,\ a_t = a] = \sum_{s'} P(s' \mid s, a)\, V^\pi(s')$$

4.5 Step 5: Assemble the Full Bellman Equation

Putting everything from Steps 3 and 4 together:

$$\boxed{V^\pi(s) = \sum_a \pi(a \mid s) \Big[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^\pi(s') \Big]}$$

This is the **Bellman Expectation Equation**. It expresses the value of a state as the expected immediate reward plus the discounted value of the next state — *recursively.*

Intuition: The value of being in state s equals the average reward you collect immediately, plus the discounted value of wherever you end up — averaged over all actions your policy takes and all transitions the environment makes.

4.6 Step 6: Bellman Optimality Equation

For the **optimal policy** π^*, instead of averaging over actions weighted by the policy, we take the **best possible action** at every state:

$$V^*(s) = \max_a \Big[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^*(s') \Big]$$

Similarly, the **optimal Q-function** satisfies:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\ \max_{a'} Q^*(s', a')$$

Remark 4.1. The Bellman equations are the backbone of *all* RL algorithms. Value-based methods (like Q-learning) solve the optimality equation directly. Policy-based methods (like REINFORCE and PPO) use the expectation equation to evaluate a given policy during optimisation.

4.7 Step 7: Policy-Based Methods — Objective Function

For policy-based methods, the objective is defined directly over trajectories:

$$J(\theta) = \sum_{\tau} p(\tau \mid \pi_\theta)\, R(\tau) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

where:

$$R(\tau) = \sum_t \gamma^t r_t \qquad \text{and} \qquad p(\tau \mid \pi_\theta) = p_0(s_0) \prod_t P(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_t \mid s_t)$$

We then want:

$$\theta^* = \arg\max_\theta \; J(\theta)$$

Instead of gradient *descent*, we perform **gradient ascent**:

$$\theta_{k+1} \leftarrow \theta_k + \alpha \cdot \nabla_\theta J(\theta_k)$$

The gradient $\nabla_\theta J(\theta)$ is called the **Policy Gradient**, and algorithms that use it are called **Policy Gradient Algorithms**.

5 Big Picture: Alignment Pipeline

$$\text{Base Pretrained LLM} \xrightarrow{\text{next-token training}} \text{SFT Model} \xrightarrow{\text{RLHF / DPO}} \text{Aligned Model}$$

1. **Pretraining**: Train on huge data for next-token generation.
2. **Supervised Fine-Tuning (SFT)**: Fine-tune on high-quality instruction data to steer towards human intent.
3. **RLHF — Preference Optimisation**: Use a reward model + policy optimisation (PPO) [3].
4. **DPO**: Directly optimise the reward without RL at all, using the Bradley–Terry model [1].

6 Reward Model

6.1 Dataset Construction

A dataset is compiled by humans comprising:

$$\{(x, y_w, y_l)\}$$

where x is the prompt, y_w is the *preferred* (good) answer, and y_l is the *rejected* (bad) answer.

6.2 Reward Objective

The reward model is trained to identify which answer is better (or assign a scalar score). The objective maximises the log-likelihood under the **Bradley–Terry model 13.2**:

$$\mathcal{L}_R = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

where $\sigma(\cdot)$ is the sigmoid function and r_ϕ is the learned reward function.

7 RLHF: Policy Gradient Optimisation — REINFORCE

Refer to Paper [5] for more information.

7.1 Objective

The goal in RL is to select a policy that maximises expected return:

$$\pi^* = \arg\max_\pi J(\pi)$$

where

$$J(\pi) = \sum_\tau p(\tau \mid \pi)\, \mathbb{E}_{\tau\sim\pi}[R(\tau)]$$

and

$$R(\tau) = \sum_t \gamma^t r_t$$

The language model is already a policy π_{θ_0} (the SFT model). We want to optimise its parameters θ so as to maximise the cumulative reward from the reward model.

7.2 Policy Gradient Theorem

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p(\tau \mid \theta)\, R(\tau)\, d\tau \\
&= \int p(\tau \mid \theta)\, \nabla_\theta \log p(\tau \mid \theta)\, R(\tau)\, d\tau \\
&= \mathbb{E}_{\tau\sim\pi_\theta}[\nabla_\theta \log p(\tau \mid \theta)\, R(\tau)] \qquad (1)
\end{aligned}$$

Since $p(\tau \mid \theta) = p_0(s_0) \prod_t P(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_t \mid s_t)$, only the policy terms depend on θ:

$$\nabla_\theta \log p(\tau \mid \theta) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Therefore:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\right]$$

In the **REINFORCE** algorithm we use a Monte-Carlo sample estimate:

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t$$

8 RLHF: Variance Reduction Techniques

8.1 First Problem with Gradient Policy Optimisation: High Variance

Recall that the REINFORCE gradient estimator is:

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t$$

This $\hat{g}$ is only an **approximation** of the true gradient $\nabla_\theta J(\theta)$:

$$\hat{g} \approx \nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\right]$$

More precisely:

$$\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot R(\tau) \approx \underbrace{\mathbb{E}[R(\tau)]}_{\text{population mean}}$$

The issue is that with a finite sample we only have access to the **sample mean**, not the population mean. By the Central Limit Theorem, the sample mean converges to the population mean, but with finite samples the variance can be **extremely large**:

$$\underbrace{\hat{g}}_{\text{sample mean}} \xrightarrow{\text{CLT}} \underbrace{\mathbb{E}[R(\tau)]}_{\text{population mean}}$$

The objective then becomes: **reduce variance without increasing sample size**.

8.2 Technique 1: Rewards-to-Go

In the gradient calculation, at every step, the reward is calculated using *all* previous steps (basically cumulative rewards). However, we **cannot change past rewards** anyway, and past reward terms cancel out in expectation. So instead we only use **future rewards** from time t onwards.

Concrete Example ($T = 4$, $t = 3$): The current gradient estimate uses:

$$\hat{g} = \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \cdot \underbrace{\left(r_{t=1} + \gamma r_{t=2} + \gamma^2 r_{t=3}\right)}_{\text{all rewards, including past}}$$

At step $t = 3$, we only need rewards from $t = 3$ onwards. The past terms $r_{t=1}$ and $r_{t=2}$ cancel in expectation (proof: they are independent of the action taken at $t = 3$), so we write:

$$G_3 = r_{t=3} + \gamma r_{t=4}$$

Proving that past terms cancel: consider the contribution of $r_{t=1}$ to the gradient at step $t = 3$:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_3 \mid s_3) \cdot r_1] = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_3 \mid s_3)] \cdot \mathbb{E}[r_1] = 0$$

since $\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$ for any fixed s.

The new **rewards-to-go** loss therefore becomes:

$$\hat{g} = \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \cdot \underbrace{\sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}}_{\text{rewards-to-go}}$$

8.3 Technique 2: Baseline Subtraction

We can further reduce variance by introducing a **baseline** $b(s_t)$, subtracted from the rewards-to-go, *without introducing bias*:

$$\hat{g} = \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \cdot \Big(\underbrace{\sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}}_{\text{rewards-to-go}} - \underbrace{b(s_{i,t})}_{\text{baseline}} \Big)$$

A natural choice for the baseline is $b(s_t) = V(s_t)$, the **state-value function**. This is a constant of the state (not of the action), so it does not bias the gradient in expectation.

8.4 Advantage Function

Definition 8.1 (Advantage Function)**.**

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

- $Q(s_t, a_t)$: expected return after taking action a_t in state s_t.
- $V(s_t)$: *average* expected return at state s_t, over all possible actions.
- $A(s_t, a_t)$: how much better action a_t is relative to the average.

Intuition: $Q(s_t, a_t)$ is the reward given to a particular state s_t for an action a_t. But if we sum over all such (s_t, a_t) pairs for all possible actions in that state, we basically get $V(s_t)$ — the average value of that state. Hence, for a given state, computing $Q(s_t, a_t) - V(s_t)$ makes us more likely to penalise a particular action **irrespective of the state we are in.** $V(s_t)$ thus becomes the baseline b, and helps in reducing variance because it accounts for the reward inclusively.

We call $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$ the **advantage term**.

Combining everything, the gradient estimate becomes:

$$\boxed{\nabla_\theta J(\theta) \approx \hat{g} = \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \cdot A(s_{i,t}, a_{i,t})}$$

8.5 Deriving the Reduced Form of $A(s,t)$: TD Residual

Recall:

$$Q(s_t, a_t) = \mathbb{E}_\pi[G_t \mid s_t = s,\ a_t = a] \qquad V(s_t) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

So the advantage expands as:

$$\begin{aligned} A(s_t, a_t) &= \mathbb{E}_\pi\big[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \;\big|\; s_t, a_t\big] - V(s_t) \\ &= \mathbb{E}_\pi\big[r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \cdots) \;\big|\; s_t, a_t\big] - V(s_t) \\ &= \mathbb{E}_\pi\big[r_t + \gamma V(s_{t+1}) \;\big|\; s_t, a_t\big] - V(s_t) \end{aligned} \tag{2}$$

If we **remove the expectation** and just consider a single sampled transition (s_t, a_t, r_t, s_{t+1}), we write the **one-step TD residual**:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

This is useful in getting $A^{(1)}$ from $A^{(k)}$:

$$A^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t$$

$$A^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) = \delta_t + \gamma \delta_{t+1}$$

8.6 k-Step Advantage Generalisation

A more generic form of the advantage for a k-step lookahead is:

$$\begin{aligned} A^{(k)}(s_t, a_t) &= \mathbb{E}\big[r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}) \;\big|\; s_t, a_t\big] - V(s_t) \\ &= \delta_t + \gamma \delta_{t+1} + \cdots + \gamma^{k-1} \delta_{t+k-1} = \sum_{j=0}^{k-1} \gamma^j \delta_{t+j} \end{aligned} \tag{3}$$

- If k (length of trajectory lookahead) is **small**: bias could be **high**, as the result depends heavily on the value function's accuracy.
- If k is **large**: variance can be very high because of the added/accumulated noise from the environment.

8.7 Generalised Advantage Estimation (GAE)

[2]

In order to perform a **bias–variance trade-off**, GAE defines the advantage function as an **exponential moving average** of all k-step estimates:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda) \sum_{k=0}^{\infty} \lambda^k \, A_t^{(k+1)}$$

Expanding:

$$\begin{aligned} \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} &= (1-\lambda)\Big[A_t^{(1)} + \lambda A_t^{(2)} + \lambda^2 A_t^{(3)} + \cdots\Big] \\ &= (1-\lambda)\Big[\delta_t + \lambda(\delta_t + \gamma\delta_{t+1}) + \lambda^2(\delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2}) + \cdots\Big] \\ &= (1-\lambda)\Big[\delta_t(1 + \lambda + \lambda^2 + \cdots) + \gamma\delta_{t+1}(\lambda + \lambda^2 + \cdots) + \gamma^2\delta_{t+2}(\lambda^2 + \cdots) + \cdots\Big] \\ &= (1-\lambda)\Big[\delta_t \cdot \frac{1}{1-\lambda} + \gamma\delta_{t+1} \cdot \frac{\lambda}{1-\lambda} + \gamma^2\delta_{t+2} \cdot \frac{\lambda^2}{1-\lambda} + \cdots\Big] \\ &= \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots \end{aligned} \tag{4}$$

$$\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}}$$

where $\lambda \in [0, 1]$ controls the bias–variance trade-off:

- $\lambda \to 0$: $\hat{A}_t \approx \delta_t$ — low variance, but high bias (relies heavily on value function accuracy).
- $\lambda \to 1$: $\hat{A}_t \approx \sum_l \gamma^l \delta_{t+l}$ — low bias, but high variance from accumulated noise.

The final gradient update using GAE is therefore:

$$\nabla_\theta J(\theta) \approx \hat{g} = \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \cdot \hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)}$$

9 Off-Policy Learning

9.1 Second Problem with Gradient Policy Optimisation: On-Policy Learning

Recall the expectation in the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[\cdots]$$

This means data **must** be collected using the *current* policy π_θ. So, every time we update the policy, old trajectories become **invalid** and we must collect new trajectories. Old data becomes useless, and hence on-policy learning is **extremely sample-inefficient**:

$$\text{collect data} \longrightarrow \text{update policy} \longrightarrow \text{discard data} \longrightarrow \text{collect data} \longrightarrow \cdots$$

We need to throw away millions of samples each iteration — this is why it is expensive.

9.2 Motivation for Off-Policy Learning

The idea is to **reuse past trajectories**. The trick that enables this is **importance sampling**:

$$\mathbb{E}_{\tau \sim p}[f(\tau)] = \mathbb{E}_{\tau \sim q}\left[\frac{p(\tau)}{q(\tau)} f(\tau)\right]$$

We have the off-policy gradient:

$$\nabla_\theta L(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\mathrm{old}}}\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, A_t\right]$$

Define the **importance ratio**:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\mathrm{old}}(a_t \mid s_t)}$$

So the off-policy surrogate objective becomes:

$$L(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\mathrm{old}}}[r_t(\theta) \, A_t] \approx \sum_{i,t} \frac{\pi_\theta(a_{i,t} \mid s_{i,t})}{\pi_{\mathrm{old}}(a_{i,t} \mid s_{i,t})} A_{i,t}$$

and its gradient:

$$\nabla_\theta L(\theta) = \mathbb{E}[r_t(\theta) \, \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, A_t]$$

9.3 Off-Policy Learning Workflow in the RLHF Setting

1. Sample trajectories (sequences of states and actions) using the **offline policy** π_{offline}.
2. Calculate rewards and advantage terms for all state–action pairs (s, a).
3. Set $\pi_{\text{offline}} = \pi_{\text{online}}$ and $\mathcal{D}_{\text{offline}} = \mathcal{D}_{\text{online}}$.
4. After k epochs:
   (a) Take a mini-batch of trajectories.
   (b) Run gradient ascent to train the online policy via $\mathcal{L}(\pi_{\text{online}}, \pi_{\text{offline}})$.

10 Trust Region Policy Optimisation (TRPO)

TPRO Paper [4]

10.1 Problems with Off-Policy Learning

When we run gradient ascent on the online policy, taking a step that is **too large in weight space** leads to excessive variance and causes **performance collapse**. The data we are training on is gathered from the old policy, and advantage calculations are only accurate if the new policy behaves *somewhat similarly* to the old one.

Hence, we need to ensure our policy updates improve steadily without accidentally destroying the model.

10.2 Trust Region Intuition

Definition 10.1 (Trust Region)**.** A **trust region** is a mathematical bubble around the old policy inside which we trust our advantage estimates, and which strictly limits how far the new policy is allowed to step outside that bubble.

10.3 TRPO Objective

Consider the following objective for off-policy learning, letting $\pi_{\text{offline}} = \pi_{\text{old}}$ and $\pi_{\text{online}} = \pi_\theta$:

$$L(\theta) = \mathbb{E}_{(s,a)\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A_t\right] \equiv \sum_{i,t} \frac{\pi_\theta(a_{i,t} \mid s_{i,t})}{\pi_{\text{old}}(a_{i,t} \mid s_{i,t})} A_{i,t}$$

The gradient is (as seen before):

$$\nabla_\theta L(\theta) = \mathbb{E}[r_t(\theta)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A_t]$$

TRPO formalises the "small step" idea via the KL divergence between old and new policies. TRPO defines the constrained optimisation:

$$\max_\theta \; L(\theta)$$
$$\text{subject to} \quad \bar{D}_{\text{KL}}(\pi_{\text{old}}, \theta) = \mathbb{E}_{s\sim\pi_{\text{old}}}[D_{\text{KL}}(\pi_{\text{old}}(\cdot \mid s) \parallel \pi_\theta(\cdot \mid s))] \;\leq\; \delta$$

where the expectation over states s_t is estimated by averaging over states in the batch.

Intuition: Maximise the same importance-weighted advantage objective we already have, but restrict how far the entire distribution is allowed to move per state.

10.4 Full Derivation of the TRPO Update Step

Let $\theta = \theta_{\text{old}} + \Delta\theta$. TRPO uses **local (quadratic) approximations** around θ_{old}:

10.4.1 Step 1: First-Order Expansion of L

$$L(\theta_{\text{old}} + \Delta\theta) \approx L(\theta_{\text{old}}) + g^\top \Delta\theta \qquad \text{where } g := \nabla_\theta L(\theta_{\text{old}})$$

We want to maximise this linear term $g^\top \Delta\theta$.

10.4.2 Step 2: Second-Order Approximation of the KL Constraint

Around θ_{old}:

$$D_{\text{KL}}(\theta_{\text{old}},\ \theta_{\text{old}} + \Delta\theta) \approx \frac{1}{2}\,\Delta\theta^\top H\,\Delta\theta$$

where H is the **Fisher Information Matrix (FIM)**:

$$H = \nabla_\theta^2 D_{\text{KL}}\big|_{\theta=\theta_{\text{old}}}$$

So the constraint becomes:

$$\frac{1}{2}\,\Delta\theta^\top H\,\Delta\theta \ \le\ \delta$$

10.4.3 Step 3: Lagrangian Formulation

Combine the objective and the constraint into a Lagrangian:

$$\mathcal{L}(\Delta\theta,\ \lambda) = g^\top \Delta\theta - \lambda\left(\frac{1}{2}\,\Delta\theta^\top H\,\Delta\theta - \delta\right)$$

10.4.4 Step 4: Taking the Gradient and Setting to Zero

$$\nabla_{\Delta\theta}\,\mathcal{L}(\Delta\theta, \lambda) = g - \lambda H\,\Delta\theta = 0$$

$$\implies \quad \lambda H\,\Delta\theta = g \quad \implies \quad \Delta\theta = \frac{1}{\lambda}\,H^{-1} g$$

10.4.5 Step 5: Solving for λ via the Equality Constraint

Substituting $\Delta\theta = \frac{1}{\lambda} H^{-1} g$ into the equality constraint $\frac{1}{2}\Delta\theta^\top H\,\Delta\theta = \delta$:

$$\frac{1}{2}\left(\frac{1}{\lambda}\right)^2 g^\top H^{-1} H\,H^{-1} g = \delta$$

$$\frac{1}{2\lambda^2}\,g^\top H^{-1} g = \delta$$

$$\lambda^2 = \frac{g^\top H^{-1} g}{2\delta}$$

$$\implies \quad \lambda = \sqrt{\frac{g^\top H^{-1} g}{2\delta}}$$

10.4.6 Step 6: Final TRPO Update Step

Substituting λ back:

$$\Delta\theta_{\text{TRPO}} = \frac{1}{\lambda} H^{-1} g = \frac{H^{-1} g}{\sqrt{\frac{g^\top H^{-1} g}{2\delta}}}$$

$$\boxed{\Delta\theta_{\text{TRPO}} = \sqrt{\frac{2\delta}{g^\top H^{-1} g}}\, H^{-1} g}$$

Remark 10.1. While TRPO guarantees monotonic improvement in theory, solving this constrained optimisation problem in practice requires calculating complex **second-order derivatives** via the Fisher Information Matrix H. This is computationally expensive and difficult to implement alongside large neural networks.

11 Proximal Policy Optimisation (PPO)

PPO Paper [3]

11.1 Motivation

PPO captures the stability of TRPO without the massive computational overhead. Instead of using the complex second-order constraint (Fisher Information Matrix), PPO uses a "trust region" enforced via a **first-order clipping mechanism**, and optimises using plain SGD over small epochs on the same batch.

11.2 PPO Clipped Objective

The PPO clipped objective is defined as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\, A_t,\ \ \text{clip}(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\, A_t)]$$

where the importance ratio is:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

11.3 Intuition and Full Case Analysis

Case 1: $A_t > 0$ (good action — we want r_t to be large).

- If $r_t \leq 1+\varepsilon$: both terms are equal, $\min(r_t A_t,\ (1+\varepsilon)A_t) = r_t A_t$. Gradient still flows normally.
- If $r_t > 1+\varepsilon$: $\text{clip}(r_t) = 1+\varepsilon$, so:

  $$\min(r_t A_t,\ (1+\varepsilon)A_t) = (1+\varepsilon)\, A_t$$

  This is **constant** in θ. There is no gradient incentive to push r_t even further beyond $1+\varepsilon$, preventing excessively large policy updates for good actions.

Case 2: $A_t < 0$ (bad action — we want r_t to be small).

- If $r_t \geq 1 - \varepsilon$: both terms are equal, $\min(r_t A_t,\ (1-\varepsilon)A_t) = r_t A_t$ (both negative). Gradient flows normally.
- If $r_t < 1 - \varepsilon$: $\text{clip}(r_t) = 1 - \varepsilon$, so:

$$\min(-r_t\,|A_t|,\ -(1-\varepsilon)|A_t|) = -(1-\varepsilon)\,|A_t|$$

  Again **constant** in θ. PPO prevents the algorithm from over-punishing already disfavoured actions within a single batch update.

Summary: Clipping Behaviour.

$$\text{clip}(r_t,\ 1-\varepsilon,\ 1+\varepsilon) = \begin{cases} 1-\varepsilon & r_t \leq 1-\varepsilon \\ r_t & 1-\varepsilon < r_t < 1+\varepsilon \\ 1+\varepsilon & r_t \geq 1+\varepsilon \end{cases}$$

Basically, we make sure **not** to move the parameters (action probabilities) too much — we want to take small, stable steps; not too far from the original policy. PPO prevents the algorithm from over-punishing already disfavoured actions within single batch updates.

12 Reward Hacking in RLHF

12.1 The Problem

In RLHF, we do *not* optimise the true human utility — we optimise a **learned reward model**. The gap between the two is where reward hacking lives.

Definition 12.1 (Reward Hacking)**.** Let trajectory $\tau = (s_0, a_0, \ldots, s_T)$ (for language models, actions are token generations). Define:

- $U(\tau)$: the true (latent) human utility — **unknown**.
- $R_\phi(\tau) = \sum_t R_\phi(s_t, a_t)$: the learned reward model.

We train a policy:

$$\hat{\pi} = \arg\max_{\pi}\ \mathbb{E}_\pi[R_\phi(\tau)]$$

But the policy we *wish* we had (the ideal policy) is:

$$\pi^* = \arg\max_{\pi}\ \mathbb{E}_\pi[U(\tau)]$$

The **reward hacking gap** under the learned optimal policy $\hat{\pi}$ is:

$$\Delta_{\text{hack}} = \mathbb{E}_{\hat{\pi}}[R_\phi(\tau) - U(\tau)]$$

Reward hacking occurs when Δ_{hack} is large and positive.

12.2 Reward Model Exploitation

Since $U(\tau)$ is unknown, we cannot calculate Δ_{hack} directly. Algorithms instead try to prevent the policy from moving into regions where R_ϕ is unreliable.

Suppose:

$$R_\phi(\tau) = U(\tau) + \varepsilon(\tau)$$

Then hacking occurs when the optimiser drives the policy towards trajectories with **large positive error** ε:

$$\mathbb{E}_{\hat{\pi}}[\varepsilon(\tau)] > 0 \quad \text{while} \quad \mathbb{E}_{\hat{\pi}}[U(\tau)] \text{ stays the same or decreases.}$$

12.3 Refined RLHF Objective to Tackle Reward Hacking

This involves two things:

1. Optimise the proxy reward $R_\phi(\tau)$.
2. Regularise against exploitation via a KL term: $D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})$.

The template objective function looks like:

$$\max_{\pi_\theta} \; \mathbb{E}_{\tau\sim\pi_\theta}[R_\phi(\tau) - \beta\, U(\tau)] - \lambda\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

The **standard KL-RLHF objective** defines a per-step reward:

$$\tilde{r}_t = R_\phi(s_t, a_t) - \lambda \log \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{ref}}(a_t \mid s_t)}$$

The full objective then takes the shape:

$$\begin{aligned}
\mathcal{L}(\theta) &= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \tilde{r}_t\right] - \lambda\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t R_\phi(s_t, a_t)\right] - \lambda\left[\sum_t \log \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{ref}}(a_t \mid s_t)}\right] - \lambda\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) \\
&= \mathbb{E}_{s\sim d^{\pi_\theta}}[\mathbb{E}_{a\sim\pi_\theta}[\tilde{r}(s,a)]] - \lambda\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) \qquad (5)
\end{aligned}$$

12.4 Plugging into PPO

PPO needs the advantage A_t; we use the advantage computed from $\tilde{r}_t$ instead of r_t. So structurally the PPO objective remains unchanged — we simply replace the reward with the KL-penalised reward $\tilde{r}_t$.

Remark 12.1. "PPO + KL penalty" is the common first-line defence against reward hacking in RLHF.

13 Direct Preference Optimisation (DPO)

DPO Paper [1]

13.1 Motivation

DPO is *not* RLHF: it directly optimises the reward function without any RL, addressing reward hacking by design.

13.2 Bradley–Terry Model

We connect preferences to a latent score $r^*(x, y)$ via the **Bradley–Terry model**:

$$P(y_w \succ y_l \mid x) = \frac{e^{r^*(x,y_w)}}{e^{r^*(x,y_w)} + e^{r^*(x,y_l)}} = \sigma(r^*(x, y_w) - r^*(x, y_l))$$

We want to maximise this probability; the MLE objective is:

$$\mathcal{L}_{\text{BT}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

13.3 Deriving the Analytical Solution to the RLHF Objective

13.3.1 Step 1: Expand the Objective

Start with the KL-regularised RLHF objective:

$$\max_{\pi_\theta} \ \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(y|x)}[r(x, y)] - \beta\, D_{\text{KL}}(\pi_\theta(y \mid x) \parallel \pi_{\text{ref}}(y \mid x))$$

Expanding the KL divergence:

$$= \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(y|x)}\left[r(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\right] \tag{6}$$

Factor out $-\beta$:

$$= -\beta\, \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(y|x)}\left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \frac{1}{\beta}\, r(x, y)\right] \tag{7}$$

Now, note that $\frac{1}{\beta}\, r(x, y)$ inside the log can be absorbed into the denominator as $\exp\left(\frac{1}{\beta} r(x, y)\right)$:

$$= -\beta\, \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(y|x)}\left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{1}{\beta} r(x, y)\right)}\right]$$

Since we are multiplying by $-\beta$ and maximising, this is equivalent to **minimising**:

$$\boxed{\min_{\pi_\theta} \ \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{y\sim\pi_\theta(y|x)}\left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{1}{\beta} r(x, y)\right)}\right]}$$

13.3.2 Step 2: Define an Unnormalised Density

This expression inside the log looks almost like a KL divergence, except the denominator does **not** integrate to 1. We fix this by defining an unnormalised density:

$$\tilde{\pi}^*(y \mid x) := \pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right)$$

and a **partition function** (normalisation constant):

$$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right)$$

The properly normalised distribution is:

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right)}{Z(x)}$$

so that:

$$\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right) = Z(x) \cdot \pi^*(y \mid x)$$

Substituting back into the minimisation objective:

$$\min_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}} \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y \mid x)}{Z(x) \cdot \pi^*(y \mid x)}\right]$$

Splitting the log:

$$= \min_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}} \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y \mid x)}{\pi^*(y \mid x)} - \log Z(x)\right]$$

$$= \min_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}}[D_{\text{KL}}(\pi_\theta(y \mid x) \parallel \pi^*(y \mid x)) - \log Z(x)]$$

13.3.3 Step 3: Apply Gibbs' Inequality

Since $Z(x)$ does **not** depend on π_θ, and by **Gibbs' inequality**:

$$D_{\text{KL}}(\pi_\theta \parallel \pi^*) \geq 0$$

with equality **if and only if** $\pi_\theta = \pi^*$. Therefore the minimum is achieved at:

$$\boxed{\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right)}{Z(x)}}$$

Remark 13.1. Note that $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \cdot \exp\left(\frac{r(x,y)}{\beta}\right)$ is summed over **all** y, which is extremely expensive to compute in practice. Hence we do **not** practically know $\pi^*(y \mid x)$ directly — but we can *assume* we know it and proceed to recover $r(x,y)$ from it.

13.4 Recovering $r(x, y)$ and Deriving the DPO Loss Function

Even though we cannot compute $\pi^*(y \mid x)$ directly, we **assume** we know it and work backwards to recover $r(x, y)$. Starting from the analytical solution:

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) \cdot \exp\left(\dfrac{r(x, y)}{\beta}\right)}{Z(x)}$$

Taking logarithms of both sides and rearranging for $r(x, y)$:

$$\log \pi^*(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{r(x, y)}{\beta} - \log Z(x)$$

$$\boxed{r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)}$$

We now substitute this expression for $r(x, y)$ back into the **Bradley–Terry model**:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

$$= \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \cancel{\beta \log Z(x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} - \cancel{\beta \log Z(x)}\right)$$

The $\beta \log Z(x)$ terms cancel exactly, leaving:

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

Parameterising π^* by θ and taking the negative log-likelihood over the preference dataset $\mathcal{D}$, the **DPO loss function** is:

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta;\, \pi_{\text{ref}}) = -\mathbb{E}_{(x,\, y_w,\, y_l) \sim \mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]}$$

Remark 13.2. DPO is **not** RLHF. The $Z(x)$ terms cancel exactly, so the intractable partition function never needs to be computed. No separate reward model is trained, and reward hacking is addressed by construction.

14 Summary: RLHF vs. DPO

Aspect	**RLHF (PPO)**	**DPO**
Reward model	Explicitly trained	Implicit in policy
RL required?	Yes (PPO/TRPO)	No
Reward hacking	Possible; mitigated by KL penalty	Addressed by construction
Optimisation	Policy gradient (PPO)	Supervised (cross-entropy-like)
Objective	$\max \mathbb{E}[r] - \beta D_{\text{KL}}$	$\min \mathcal{L}_{\text{DPO}}$

References

[1] Rafael Rafailov et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *arXiv preprint arXiv:2305.18290* (2023).

[2] John Schulman et al. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *arXiv preprint arXiv:1506.02438* (2015).

[3] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[4] John Schulman et al. "Trust Region Policy Optimization". In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 1889–1897.

[5] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8 (1992), pp. 229–256.